Cooking It Up | CodeMonks

COP3530 Project 2b - Group Report | November 8, 2026

Team: Artem Antonian (GitHub: B3ar1000); Nathan Chin-Lue (GitHub: natecl); Marissa Micioni (GitHub: marissamicioni2025-hub).

GitHub repository: https://github.com/marissamicioni2025-hub/Cooking-It-Up

Video demo URL: https://youtu.be/tYfpW3INpe0

Extended and refined proposal: problem and motivation

Finding a recipe that fits the ingredients already available can require repeatedly comparing long ingredient lists. A user may find an appealing recipe but still need to determine which ingredients are missing. Cooking It Up addresses this problem by accepting a pantry list, searching a large recipe collection, and displaying ranked matches with missing ingredients.

The project connects a practical meal-search task to a data-structures comparison. Jaccard and binary cosine similarity use the same ingredient sets but normalize their overlap differently. Implementing both exposes how the choice of similarity measure can affect recommendations and execution time.

Implemented user-facing features

The command-line program loads the Food.com recipe CSV and reports the number of usable recipes, vocabulary size, and loading time. Its menu supports a new search or exit. For each search, users enter comma-separated ingredient phrases, choose a positive result count, and select Jaccard, cosine, or both. The program displays recipe names, similarity scores, overlap counts, missing ingredients, and timed query execution.

Users can search repeatedly and run benchmarks after a search. The menu retries invalid numeric selections and handles end-of-file. Empty queries are rejected. Unknown phrases are reported and excluded from both similarity methods and benchmarks; queries with no recognized ingredients return to the menu. Normalization standardizes ASCII case and whitespace and removes duplicate phrases.

Implementation: data, algorithms, and responsibilities

Dataset and representation

The project uses RAW_recipes.csv from Food.com Recipes and Interactions on Kaggle [2], associated with the recipe dataset of Majumder et al. [3]. The tested file contains 231,637 raw records. Loading retains 231,636 recipes and rejects one blank-name record (ID 368257), exceeding the proposed minimum of 100,000. The cleaned vocabulary contains 14,941 ingredient phrases. Ratings and user-interaction records are not used.

DataLoader.cpp streams quoted and multiline CSV records and parses ingredient lists as strings. It validates positive recipe IDs, rejects unusable records, removes duplicate IDs, normalizes ingredient phrases, and removes repeated ingredients. A shared unordered_map maps phrases to integer IDs. Each Recipe stores its original ID and name plus a sorted unique vector of ingredient IDs. Query objects use this same vocabulary. A failed reload preserves the previously loaded database.

Algorithms and supporting data structures

Both similarity methods are implemented directly in C++ using a two-pointer overlap traversal. For recognized pantry set Q and recipe set R, Jaccard is |Q intersection R| / (|Q| + |R| - |Q intersection R|) [4]. Binary cosine is |Q intersection R| / sqrt(|Q| * |R|) [5]. For Q={chicken,rice,salt} and R={chicken,rice,garlic,onion}, Jaccard is 2/5=0.4 and cosine is 2/sqrt(12), approximately 0.577.

Scoring creates a vector of positive-overlap ScoredRecipe records. Ranking.cpp implements a custom bounded min-heap for top-k selection and a custom merge sort for ordering the winners or all candidates. Scores within 1e-9 are treated as ties, followed by more matched ingredients, fewer recipe ingredients, and lower recipe ID. Vectors store recipes and ingredient IDs; a hash set detects duplicate recipe IDs. Missing-ingredient display checks each recipe ingredient against the pantry IDs.

Tools and completed responsibilities

The application uses C++17 and the standard library, including vectors, unordered containers, std::chrono::steady_clock, and sqrt. The documented build uses g++. setup_data.sh uses Python, pip, and kagglehub to retrieve the large dataset separately. GitHub supports branches, commits, pull requests, and integration.

Nathan handled recipe/data structures, loading and cleaning, vocabulary construction, Jaccard, tests, and report assembly. Artem handled cosine, custom ranking, top-k selection, tie rules, associated tests, and the group reflection. Marissa handled the menu, result/missing-ingredient display, benchmarking, and final integration. The team coordinated interfaces, debugging, and the completed presentation.

Analysis: design changes and worst-case complexity

Changes from the original proposal

The implementation retains the proposed dataset, binary ingredient sets, C++ interface, and similarity comparison. A single Find recipes flow collects the pantry, result count, and method for each search. Added custom heap selection, merge sort, tie rules, tests, and a dataset setup script support efficient top-k selection, repeatable ordering, and validation. Final integration standardized both methods and benchmarks on recognized ingredients for a consistent comparison.

Worst-case bounds

Let N be recipe records processed, M ingredient memberships, V vocabulary size, q query cardinality, r maximum ingredients per recipe, P positive matches, K requested results, k=min(K,P), T CSV characters, and S a benchmark prefix size. The table treats bounded ingredient/name lengths as constant; string hashing, comparisons, and copies additionally scale with their lengths. Hash-collision worst cases are included.

Major function / feature	Worst-case time
CSV scanning	O(T)
Loading, cleaning, vocabulary	O(T + N r log r + M V + N^2)
Query normalization and lookup	O(q log q + q V)
Two-pointer overlap, one recipe	O(q + r)
Either score after overlap	O(1)
All-recipe scoring	O(N(q + r))
Custom full merge sort	O(P log(P+1))
Heap selection and winner sort	O(P log(k+1) + k log(k+1))
Result lookup / missing display	O(k N + k r q)
One scoring-only benchmark	O(S(q + r))

Expected hash-table behavior makes vocabulary lookup and duplicate-ID checks constant-probe operations; adversarial collisions produce the larger loading/query bounds above. Recipe ingredient sorting enables linear overlap traversal. Both similarity methods share the same asymptotic scoring cost. Full merge sort needs O(P) auxiliary records; the heap holds O(k) candidates but reserves O(K) capacity. The current search first materializes all P positive scores, so the scored-result vector uses O(P) storage; including reserved heap capacity, search-result storage is O(P+K).

The database uses O(N+M+V) structural storage plus strings and allocation overhead. Query storage is O(q). The missing-ingredient display scans the recipe collection to find each selected ID and uses linear pantry membership checks; these display costs are outside the printed query timer.

Analysis: validation and measured performance

Correctness checks

The integrated program compiled with C++17 and compiler warnings enabled. NathanTests.cpp passed all 2,033 checks, including parsing, cleaning, failed-reload preservation, and 1,000 randomized set-reference comparisons. Tests_Artem.cpp passed its three groups covering cosine examples, empty/zero-overlap inputs, ties, and top-k edge cases. Full-data checks exercised both methods, missing ingredients, benchmarks, repeated searches, and the merged fix: adding an unknown phrase leaves results and scores unchanged for each method.

Performance experiment

For this report, the repository's unchanged runBenchmarks function was invoked using five all-known pantry queries, four file-prefix sizes, one warmup, and 21 timed repetitions per query. The table is the arithmetic mean of each query's median, measured in milliseconds. Tests ran on an Apple M4 Pro with 24 GiB RAM, macOS 26.6.2, Apple Clang 21, and -O2 optimization. Loading the full file took approximately 1.85 seconds in a separate observation.

Recipes	Jaccard (ms)	Cosine (ms)
1,000	0.0141	0.0139
10,000	0.1541	0.1585
50,000	0.8293	0.8750
100,000	1.6866	1.7762

The queries were chicken/rice/salt; flour/sugar/eggs/butter; tomatoes/garlic/olive oil/onion; milk; and water/salt/pepper/onion/garlic/butter/sugar/flour. The benchmark timer covers scoring only, excluding subset construction, loading, query preparation, ranking, and display. The application's query timer instead includes scored-result construction and top-k ranking. These two timings should not be interpreted as the same measurement.

Both methods increased in time with dataset size. Jaccard was slightly faster at 100,000 recipes in this experiment; this reflects constants and this workload, not a better asymptotic bound or a universal winner. Prefix composition, fixed Jaccard-before-cosine order, cache state, and system noise limit generalization. The full database's logical payload was 15.64 MiB; this excludes object, capacity, hash-node, and allocator overhead and is not process RSS.

Current limitations

Matching uses exact phrases and binary presence, without synonyms or quantities. Comma-separated input cannot directly represent ingredient phrases that contain commas. Unknown ingredients are reported and excluded consistently from searches and benchmarks. The ranking comparator uses an epsilon tie policy; floating-point boundary cases deserve further testing. Future work could improve relevance and avoid scanning every recipe.

Reflection and references

Overall Group Experience

Cooking It Up gave CodeMonks practical experience with modular C++ development and algorithm comparison. Nathan developed the dataset/Jaccard components, Artem developed cosine and ranking, and Marissa developed the interface, missing-ingredient display, and benchmarks. A shared Recipe.h contract and ingredient vocabulary connected the modules, while GitHub branches and pull requests supported collaboration.

Challenges and How We Addressed Them

Dataset cleaning: Quoted CSV fields and repeated ingredient phrases required careful parsing and normalization. Mapping phrases to shared integer IDs, sorting, and deduplication enabled O(q+r) two-pointer overlap traversal.

Tied scores: Ranking.cpp resolves scores within 1e-9 using more matched ingredients, fewer recipe ingredients, and lower recipe ID. Explicit rules make equal-score results predictable.

Top-k selection: Instead of sorting all P positive matches, the bounded heap selects k candidates in O(P log(k+1)); merge-sorting the winners adds O(k log(k+1)). Scoring still scans the recipe collection.

What We Would Change Next Time

We would plan shared interfaces earlier and add an inverted index from ingredient IDs to recipe IDs to reduce scanning. Synonym normalization could connect phrases such as scallions and green onions. Weighting ingredients could give common pantry items less influence than proteins or produce.

What each member learned

Nathan Chin-Lue: Nathan learned to scope larger tasks into functions that interact and build on one another. He strengthened his ability to test both individual functions and program behavior. Designing tests that properly verified success was his biggest initial challenge.

Artem Antonian: Artem learned cosine similarity for sparse binary ingredient vectors, the effect of floating-point precision on C++ comparators, and the efficiency of a bounded min-heap compared with sorting all candidates.

Marissa Micioni: Marissa learned to communicate across separate responsibilities, coordinate schedules, connect multiple C++ files, and collaborate through GitHub. Integrating independently developed components was challenging. She would plan shared interfaces earlier next time and gained confidence working with teammates whose coding experience she did not initially know.

References

[1] CodeMonks. Cooking It Up: source, README, and tests. Main commit b3c4698, November 8, 2026. GitHub.

[2] Shuyang Li. Food.com Recipes and Interactions, Kaggle, version 2. Dataset.

[3] B. P. Majumder et al. Generating Personalized Recipes from Historical User Preferences. EMNLP-IJCNLP, 2019. ACL Anthology.

[4] scikit-learn documentation. Jaccard similarity coefficient.

[5] scikit-learn documentation. Cosine similarity.